

Processing System (PS)

Software Design Explained in Simple Terms

PDSA-FPGA Crypto Miner - PS Software Deep Dive

Generated: 2026-06-20 12:08

ARM Cortex-A53 @ 1.2 GHz | Ubuntu 22.04 | 940 lines C

Table of Contents

1	What is the PS? (Processing System)
2	Software Architecture Overview
3	pdsa_hal.h: Hardware Abstraction Layer
4	/dev/mem and mmap: How Linux Talks to FPGA
5	pdsa_csr_map.h: The Register Address Book
6	pdsa_main.c: The Main Application
7	Startup Sequence: What Happens at Boot
8	NIST Self-Test: How It Feeds Vectors to PL
9	sha256_sw.c: Software SHA-256 Reference
10	Local Mining Loop (No Pool)
11	PDSA Evaluation: When to Switch Algorithms
12	DPR Orchestration: Triggering Reconfiguration
13	CSV Logging: Measuring Performance
14	Build System: Makefile Explained
15	Complete File Reference

1. What is the PS? (Processing System)

The PS (Processing System) is the ARM processor side of the Zynq UltraScale+ MPSoC chip. Unlike the PL (FPGA fabric) which implements custom hardware circuits, the PS runs traditional software on ARM processors.

The KV260's PS contains:

- Quad-core ARM Cortex-A53: 64-bit processor running at 1.2 GHz. This is the main application processor running Linux Ubuntu 22.04.
- Dual-core ARM Cortex-R5F: Real-time processor for low-latency tasks (not used in our design).
- 2 GB DDR4 RAM: System memory shared between PS and PL (via the memory controller).
- Peripherals: Gigabit Ethernet, USB 3.0, DisplayPort, SD card controller, UART, I2C, SPI.
- PS-PL Interfaces: AXI buses (including our AXI-Lite at 0xA0000000), interrupts, EMIO, clocks.

PS and PL in the Zynq Chip

+=====+			
	Zynq UltraScale+ MPSoC (XCZU5EV)		
	+-----+	+-----+	
	PS (Processing System)	PL (FPGA Fabric)	
	ARM Cortex-A53 x4	CLB Array	
	@ 1.2 GHz	88,590 LUTs	
		92,869 FFs	
	Runs: Linux Ubuntu	3,974 CARRY8	
	Runs: pdsa_miner (C)		
		Contains:	
	Memory: 2 GB DDR4	SHA-256d pipe	
	Storage: SD card + SSD	Keccak pipe	
		PDSA logic	
	+-----+	+-----+	
	+----- AXI-Lite @ 0xA0-----+		
	+----- IRQ GIC SPI 89 <-----+		
	+=====+		

The PS is where all the software runs. It handles: communicating with the mining pool via Ethernet, computing midstate from block headers, sending jobs to the PL via AXI-Lite, reading results, deciding when to switch algorithms, logging performance data, and the user interface.

2. Software Architecture Overview

The PS software is organized in layers, each with a specific responsibility:

LAYER	FILES	RESPONSIBILITY
-----	-----	-----
Application	pdsa_main.c	Main loop, NIST self-test, PDSA FSM, CSV logging, local mining loop
HAL	pdsa_hal.h	/dev/mem mapping, CSR read/write with memory barriers (DSB)
Register Map	pdsa_csr_map.h	Register offset definitions, bit masks (PS <-> PL shared)
SHA-256 Reference	mining/sha256_sw.c mining/sha256_sw.h	Software SHA-256 for NIST self-test verification
Block Header	mining/block_header.h	80-byte Bitcoin/Kaspa header structure (no pool parser)

Execution Flow (High Level)

```

main()
|
+--> pdsa_hal_init()           // Open /dev/mem, mmap CSR
+--> run_nist_self_test()      // Verify PL + SW SHA-256
+--> run_nist_kat()           // HW single-SHA-256 KAT
+--> run_genesis_block_test() // Bitcoin genesis double-SHA-256
+--> run_pdsa_dpr_self_test() // Verify PDSA + DPR
|
+--> create_simulated_job()    // Dummy header + easy target
+--> send_job_to_pl()         // Write to CSR, start engines
|
+--> [MAIN LOOP]
|
|   +--> Poll STATUS_FOUND
|   |
|   |   +--> If found: print nonce, restart (nonce+1)
|   |
|   +--> Every 5s: simulated PDSA eval
|   |
|   |   +--> sim_pt -= 20 (100 -> 80 -> 60 -> ...)
|   |   +--> Write CSR_PT_CURR
|   |   +--> Read CSR_PDSA_DECISION
|   |   +--> If SWITCH_PT: do_dpr_switch()
|   |   +--> create_simulated_job() for new algo
|   |   +--> send_job_to_pl()
|   |
|   +--> Every 30s: status print + watchdog check
+--> log_event()              // Write CSV log entry

```

3. pdsa_hal.h: Hardware Abstraction Layer

pdsa_hal.h is a C header file that provides the interface between the PS software and the PL hardware. It uses simple memory-mapped I/O via /dev/mem to read and write CSR registers.

What It Provides

- pdsa_hal_t struct: Holds the file descriptor and memory pointer for the mapped CSR region.
- pdsa_hal_init(): Opens /dev/mem, mmap's the physical address 0xA0000000 into user space.
- pdsa_csr_write(hal, offset, value): Writes a 32-bit value to a CSR register.
- pdsa_csr_read(hal, offset): Reads a 32-bit value from a CSR register.
- pdsa_hal_close(): Unmaps memory and closes the file descriptor.

```
// How a CSR write works (simplified):
static inline void pdsa_csr_write(pdsa_hal_t *hal,
                                uint32_t off, uint32_t val) {
    // Calculate the address in mapped memory
    volatile uint32_t *r = (volatile uint32_t*)
        ((uint8_t*)hal->base + off);
    // Write the value
    *r = val;
    // Memory barrier: force the write to complete
    __asm__ volatile("dsb sy" ::: "memory");
}

// How a CSR read works (simplified):
static inline uint32_t pdsa_csr_read(pdsa_hal_t *hal,
                                    uint32_t off) {
    volatile uint32_t *r = (volatile uint32_t*)
        ((uint8_t*)hal->base + off);
    // Memory barrier: ensure previous writes complete
    __asm__ volatile("dsb sy" ::: "memory");
    return *r;
}
```

The 'volatile' keyword tells the compiler not to optimize away the read/write (important because the register values can change outside of C code, e.g., when the PL updates status bits). The 'dsb sy' instruction is an ARMv8 data synchronization barrier that ensures all memory accesses complete before continuing.

4. /dev/mem and mmap: How Linux Talks to FPGA

/dev/mem is a Linux character device that provides access to the physical memory of the system. By opening this device and using mmap(), user-space programs can read and write physical addresses directly - including the PL's AXI-Lite registers.

Step-by-Step: How It Works

Step 1: Open /dev/mem

```
-----
int fd = open("/dev/mem", O_RDWR | O_SYNC);
// O_RDWR = read + write access
// O_SYNC = no caching (writes go directly to hardware)
```

Step 2: Map physical address into user space

```
-----
void *base = mmap(NULL,                // Let kernel choose address
                  0x10000,             // Map 64 KB
                  PROT_READ|PROT_WRITE, // Read + write
                  MAP_SHARED,          // Share with other processes
                  fd,                  // /dev/mem file descriptor
                  0xA0000000);          // Physical address of CSR
```

Step 3: Access registers via pointer

```
-----
volatile uint32_t *csr = (volatile uint32_t *)base;
csr[0] = 0x01;           // Write to CSR_CTRL (offset 0x000)
uint32_t status = csr[1]; // Read from CSR_STATUS (offset 0x004)
```

Why This Works

The KV260's memory map places the PL's AXI-Lite slave at physical address 0xA0000000 (set in the Vivado Block Design). The Zynq's memory controller routes any PS access to this address through the AXI bus to the PL. The PL's axi_lite_csr module decodes the address and reads/writes the appropriate register.

IMPORTANT: /dev/mem requires root access (sudo). If the kernel has CONFIG_STRICT_DEVMEM=y, access to addresses above 1 MB is blocked. In that case, use the UIO driver via the Device Tree Overlay (see implementation guide).

5. pdsa_csr_map.h: The Register Address Book

pdsa_csr_map.h is the shared header file that defines the address offsets for every CSR register. This file is used by BOTH the PL RTL (as a reference) and the PS software (as #include).

Why Shared?

By using a single header file, we ensure the PS and PL agree on the register layout. If we change a register address, we only need to update one file. This prevents the bug we found where the PL had different addresses than the PS expected.

```
// Example definitions from pdsa_csr_map.h:
#define CSR_CTRL            0x000 // [W] Control register
#define CSR_STATUS          0x004 // [R] Status register
#define CSR_ALGO_SEL        0x008 // [W] Algorithm select (0=BTC,1=KAS)
#define CSR_NONCE           0x00C // [W] Starting nonce
#define CSR_TARGET_BASE     0x010 // [W] 256-bit target (8 words)
#define CSR_MIDSTATE_BASE   0x030 // [W] 256-bit midstate (8 words)
#define CSR_JOBDATA_BASE    0x050 // [W] 640-bit job data (20 words)
#define CSR_RESULT_HASH_BASE 0x0C8 // [R] 256-bit result (8 words)

// Bit masks for control/status:
#define CTRL_START          0x01 // Start mining
#define CTRL_STOP           0x02 // Stop mining
#define CTRL_DECOUPLE       0x04 // Decouple RP
#define CTRL_TRIGGER_DPR    0x08 // Trigger DPR switch

#define STATUS_BUSY         0x01 // Engines running
#define STATUS_FOUND        0x02 // Nonce found
#define STATUS_DECOUPLED    0x04 // RP decoupled
#define STATUS_DPR_ACTIVE   0x08 // DPR in progress
```

6. pdsa_main.c: The Main Application

pdsa_main.c (940 lines of C) is the main application running on the KV260's ARM processor. It implements the complete standalone mining controller with no external dependencies (no pool, no network).

Key Data Structures

```
typedef enum { ALGO_BITCOIN=0, ALGO_KASPA=1 } algo_t;

typedef enum { STATE_IDLE, STATE_VERIFY, STATE_MINING,
               STATE_EVALUATING, STATE_DPR_PREPARE,
               STATE_DPR_LOADING, STATE_DPR_RESTART,
               STATE_DONE } pdsa_state_t;

typedef struct {
    pdsa_hal_t    hal;           // HAL instance
    algo_t        current_algo;  // Current algorithm
    pdsa_state_t  state;         // FSM state
    mining_job_t  current_job;   // Current mining job
    int           has_job;       // Job loaded in PL
    uint64_t      total_hashes;  // Total hash count
    uint32_t      shares_*;      // Nonce counters
    uint32_t      dpr_count;     // DPR switch count
    uint32_t      dpr_latency;   // Last DPR latency
    FILE          *csv_log;      // CSV log file
    time_t        last_eval;     // Last PDSA eval time
    uint32_t      start_nonce;   // Starting nonce value
} pdsa_ctx_t;
```

Key Functions

- send_job_to_pl(): Writes target, midstate, job_data to CSR, sets nonce and starts engines. Handles both Bitcoin (midstate) and Kaspera (full 80-byte header) paths.
- create_simulated_job(): Creates a dummy Bitcoin or Kaspera block header with easy target for standalone mining.
- read_result_hash(): Reads 8 x 32-bit words from CSR_RESULT_HASH_BASE.
- feed_nist_to_pl(): Feeds a padded NIST vector to PL for self-test.
- run_nist_self_test(): Runs all 5 NIST verification tests, displays PASS/FAIL.
- feed_nist_kat() / run_nist_kat(): Hardware single-SHA-256 Known-Answer Test (target=0 mode).
- run_genesis_block_test(): Verifies double-SHA-256 pipeline using Bitcoin genesis block.
- run_pdsa_dpr_self_test(): Tests PDSA decisions and DPR decouple/recouple.
- do_dpr_switch(): Orchestrates DPR sequence (stop, decouple, FPGA manager, recouple).
- get_hashrate(): Computes hash rate from hash_count delta over time.
- log_event(): Writes performance data to CSV file.
- sig_handler(): Handles Ctrl+C for graceful shutdown.

7. Startup Sequence: What Happens at Boot

When pdsa_main starts, it follows this sequence:

```
=====
PDSA-FPGA Mining Controller v2.0
Kria KV260 - Bitcoin + Kaspa Adaptive
(Standalone / Self-Test Mode)
=====

[INIT] Mapping PL CSR registers...
[HAL] Mapped PL CSR @ 0xA0000000
// Opens /dev/mem, mmap 0xA0000000 (64 KB)

[VERIFY] Running NIST SHA-256 self-test...
// Tests 1-2: Software SHA-256 of NIST vectors
// Tests 3-4: Feed vectors to PL, verify pipeline
// Test 5: PL responsiveness (start/stop toggles)

[KAT] Running NIST SHA-256 Known-Answer Test...
// Tests HW single-SHA-256 bypass (target=0 mode)

[GENESIS] Running Bitcoin genesis block test...
// Verifies double-SHA-256 against known hash

[PDSA-TEST] Running PDSA/DPR self-test...
// Tests CONTINUE, SWITCH_PT, SWITCH_BCV decisions
// Tests decouple/recouple mechanism

[SIM] Created simulated job ... (algo=Bitcoin)
[HAL] Sending job to PL hardware...
// Local mining loop begins (no pool needed)

[STATUS] Algo=Bitcoin Hashrate=125000000 H/s ...
// Mining loop: PDSA eval every 5s, status every 30s
```

What if Self-Test Fails?

If any self-test fails, the miner prints the failure reason and exits. This prevents running with broken hardware. Common failures:

- SW SHA-256 FAIL: sha256_sw.c has a bug (should never happen with NIST vectors).
- PL pipeline FAIL: The PL returned wrong hash. Check bitstream loading and CSR access.
- Genesis FAIL: Double-SHA-256 pipeline broken. Check midstate computation or PL design.
- PDSA decision FAIL: PDSA logic in PL is returning wrong codes.
- DPR decouple FAIL: STATUS_DECOUPLED did not toggle after writing CSR_CTRL decouple bit.

8. NIST Self-Test: How It Feeds Vectors to PL

The NIST self-test is the most important verification step. It proves the PL's hash pipeline is working correctly by feeding it known test vectors.

Test 1: Software SHA-256("abc")

The PS computes SHA-256 of the string "abc" using sha256_sw.c and compares against the known NIST hash. This verifies the software reference implementation is working.

```
Input:      "abc" (0x616263)
Expected:   BA7816BF 8F01CFEA 414140DE 5DAE2223
           B00361A3 96177A9C B410FF61 F20015AD
```

Test 2: Software SHA-256(multi-block)

Same as Test 1 but with a 56-byte message that spans 2 SHA-256 blocks.

Test 3: Feed NIST Vector "abc" to PL

The PS writes data to the PL's CSR registers and starts the engines:

```
Step 1: Write SHA-256 IV to CSR_MIDSTATE_BASE
        (0x6A09E667 BB67AE85 3C6EF372 A54FF53A
         510E527F 9B05688C 1F83D9AB 5BE0CD19)

Step 2: Write padded NIST block to CSR_JOBDATA_BASE
        (0x61626380 00000000 ... 00000018)

Step 3: Write all-FFs as target (impossible to meet,
        just testing pipeline runs)

Step 4: Write nonce=0, set CSR_CTRL start bit

Step 5: Poll CSR_STATUS until STATUS_BUSY clears
        or STATUS_FOUND asserts

Step 6: Read CSR_RESULT_HASH (8 words at 0xC8-0xE4)
        Verify: pipeline produced output (non-zero hash)

Step 7: Read CSR_GOLDEN_NONCE (0x0B8)
        Verify: pipeline found a nonce (deterministic
        for the behavioral hash model)
```

Test 4: Same as Test 3 with Multi-Block Vector

Same feed procedure but with the second NIST vector.

Test 5: PL Responsiveness

Write CSR_CTRL start -> read STATUS_BUSY should be 1. Write stop -> read STATUS_BUSY should be 0. This confirms the AXI-Lite path and CSR registers work.

9. sha256_sw.c: Software SHA-256 Reference

sha256_sw.c is a standalone software implementation of SHA-256 written in C. It is compiled and runs on the ARM processor. Its primary purpose is to provide a known-good reference for verifying the PL's hardware pipeline.

Implementation Details

- Follows FIPS 180-4 specification exactly.
- Standard C, no platform-specific optimizations (readable, verifiable).
- Handles arbitrary-length messages (not just single blocks).
- Produces bit-exact results matching NIST test vectors.
- Used by run_nist_self_test() to compute expected hashes for comparison.

```
// Function signature:
void sha256(const uint8_t *data, size_t len, uint8_t hash[32]);

// Usage in self-test:
uint8_t hash[32];
sha256((const uint8_t*)"abc", 3, hash);
// hash now contains BA7816BF 8F01CFEA...
```

The sha256_sw.c implementation is also used for double-SHA-256 (SHA-256d) which is just two calls: sha256(data, len, temp); sha256(temp, 32, hash);

10. Local Mining Loop (No Pool)

The current version runs in standalone mode with no pool connection. Instead of receiving jobs from a Stratum pool, it creates simulated mining jobs locally and runs the PL hash engines continuously for testing, benchmarking, and PDSA evaluation.

How the Local Mining Loop Works

1. CREATE SIMULATED JOB:
 - `create_simulated_job()` builds a dummy block header:
 - Bitcoin path: SHA-256 IV as midstate, merkle_tail=0x80000000, timestamp=0, nbits=0x0280
 - Kaspa path: 80-byte header filled with 0xAA pattern
 - Target: 0x00000000FFFF... (easy, $\sim 2^{48}$ difficulty)
2. SEND TO PL:
 - `send_job_to_pl()` writes to CSR registers:
 - Target, midstate (BTC), job_data, algo_sel, nonce
 - Sets CSR_CTRL start bit -> PL engines begin hashing
3. POLL FOR NONCE:
 - Read CSR_STATUS; if STATUS_FOUND asserted:
 - Read CSR_GOLDEN_NONCE and CSR_HCNT_LO/HI
 - Print found nonce and hash count
 - Clear STATUS_FOUND (CSR_CTRL stop)
 - Restart with nonce+1 (keeps searching)
4. PDSA EVALUATION (every 5s):
 - `sim_pt` decrements: 100 -> 80 -> 60 -> 40 -> 20
 - Write `sim_pt` to CSR_PT_CURR
 - Read CSR_PDSA_DECISION from PL
 - If SWITCH_PT (0x01): execute `do_dpr_switch()`
 - After DPR, create new simulated job for new algo
 - Reset `sim_pt` = 100
5. WATCHDOG (every 30s):
 - Check CSR_HCNT_LO/HI is still incrementing
 - If stalled (same value twice): recreate job
6. NONCE RANGE WRAP:
 - If `start_nonce` $\geq$ 0xFFFFF000: restart job

Why Standalone Mode?

- No network dependency: Works entirely offline for testing and development.
- Full pipeline validation: Exercises the exact same PL hash engines as pool mining.
- PDSA algorithm switching: Simulated PT decline triggers real DPR between SHA-256d and SHA-3.
- Benchmarking: Captures hashrate, nonce rate, DPR latency to CSV for research data.
- Building block for pool mode: The same `send_job_to_pl()` and nonce-handling code works unchanged with a real pool.

11. PDSA Evaluation: When to Switch Algorithms

The PS runs PDSA evaluation every 5 seconds while mining. In standalone mode, it simulates declining pool token availability to exercise the PDSA decision logic and DPR switching.

Evaluation Sequence

PDSA evaluation in standalone mode:

1. Decrement simulated PT percentage:
sim_pt -= 20 (starts at 100, goes 100->80->60->40->20)
2. Write to PL CSR:
CSR_PT_THRESH (0x0A0) = 30 (fixed threshold)
CSR_PT_CURR (0x0A4) = sim_pt
CSR_BCV_STATUS (0x0A8) = 0 (BCV not used in sim)
3. Read PDSA decision from PL hardware:
decision = CSR_PDSA_DECISION (0x0C4)
4. Act on decision:
0x00 = CONTINUE: Keep mining current algorithm
0x01 = SWITCH_PT: DPR switch to other algorithm
0x02 = SWITCH_BCV: DPR switch (triggered manually)

Example Simulated Cycle

```
Cycle 1 (t=5s): PT=80%, thresh=30% -> CONTINUE
Cycle 2 (t=10s): PT=60%, thresh=30% -> CONTINUE
Cycle 3 (t=15s): PT=40%, thresh=30% -> CONTINUE
Cycle 4 (t=20s): PT=20%, thresh=30% -> SWITCH_PT!
                |
                DPR: Bitcoin -> Kasp
                |
Cycle 5 (t=25s): PT=100% (reset), thresh=30% -> CONTINUE
(Kasp now active, sim_pt declines again from 100)
Cycle 9 (t=45s): PT=20%, thresh=30% -> SWITCH_PT!
                |
                DPR: Kasp -> Bitcoin
... repeats forever, switching between both algorithms
```

12. DPR Orchestration: Triggering Reconfiguration

When PDSA decides to switch algorithms, the PS executes `do_dpr_switch()` which orchestrates the Dynamic Partial Reconfiguration sequence.

```
do_dpr_switch(ctx, new_algo):
    1. Stop current mining:
        pdsa_csr_write(hal, CSR_CTRL, CTRL_STOP)
        // Engines stop processing

    2. Decouple RP:
        pdsa_csr_write(hal, CSR_CTRL, CTRL_DECOUPLE)
        // Static shell isolates RP inputs

    3. Verify decoupled:
        status = pdsa_csr_read(hal, CSR_STATUS)
        assert(status & STATUS_DECOUPLED)

    4. Trigger DPR (load new bitstream):
        fpga_fd = open("/sys/class/fpga_manager/fpga0/firmware")
        write(fpga_fd, "pdsa_rm_kaspa.bin")
        // ICAP reconfigures the RP region

    5. Wait for DPR to complete:
        sleep(1) // In production: poll STATUS_DPR_ACTIVE

    6. Recouple RP:
        pdsa_csr_write(hal, CSR_CTRL, 0) // Clear decouple

    7. Update algo and restart:
        ctx->algo = new_algo
        send_job_to_pl(ctx) // Send current job to new RM
```

The entire DPR switch takes approximately 10-50 ms depending on the partial bitstream size. During this time, the PS cannot mine (no RM is active). The `STATUS_DECOUPLED` bit indicates the RP is isolated, and `STATUS_DPR_ACTIVE` indicates reconfiguration is in progress.

IMPORTANT: In the current implementation, DPR is not fully functional because the ICAP controller is not instantiated in the Vivado BD and partial bitstreams have not been generated. The decouple/recouple mechanism works (verified by the self-test) but actual bitstream swapping requires additional Vivado DFX flow setup.

13. CSV Logging: Measuring Performance

The miner writes performance data to `/home/root/pdsa_measurements.csv` for post-processing and analysis. This data is used to generate the research paper's performance graphs.

CSV Format

Columns:

```
timestamp, state, algorithm, event,
hashrate_hps, hash_count, dpr_latency_cycles,
shares_acc, shares_rej, dpr_count, pl_status
```

Example row:

```
2026-06-02 12:00:00, MINING, Bitcoin, MINING_START,
125000000, 0, 0, 0, 0, 0, 0x00
```

```
2026-06-02 12:00:05, MINING, Bitcoin, NONCE_FOUND,
125000000, 500000000, 0, 1, 0, 0, 0x02
```

```
2026-06-02 12:00:10, EVALUATING, Bitcoin, PDSA_CONTINUE,
125000000, 1000000000, 0, 1, 0, 0, 0x00
```

```
2026-06-02 12:00:15, DPR, Bitcoin, DPR_START,
0, 1000000000, 0, 1, 0, 1, 0x04
```

```
2026-06-02 12:00:16, MINING, Kasper, MINING_START,
100000000, 0, 12000, 1, 0, 1, 0x00
```

Log Events

- MINING_START: New mining job sent to PL.
- NONCE_FOUND: PL found a valid nonce. Hashrate = hash_count / elapsed.
- SHARE_ACCEPTED: Pool accepted our share.
- SHARE_REJECTED: Pool rejected our share (usually stale job).
- PDSA_CONTINUE / PDSA_SWITCH: PDSA evaluation result.
- DPR_START / DPR_END: DPR switch sequence.
- POOL_DISCONNECT / POOL_RECONNECT: Network events.

14. Build System: Makefile Explained

The Makefile compiles the PS software on the KV260 itself. It uses GCC for ARM64 (aarch64).

```
# Variables
CC      = aarch64-linux-gnu-gcc    # Cross-compiler for ARM64
CFLAGS  = -O2 -Wall -Wextra -static
INCLUDES = -I. -I./mining

# pdsa_miner: standalone CSR test tool
TARGET1  = pdsa_miner
SRCS1    = pdsa_miner.c pdsa_csr.c

# pdsa_main: main PDSA mining controller
TARGET2  = pdsa_main
SRCS2    = pdsa_main.c mining/sha256_sw.c

all: $(TARGET1) $(TARGET2)
```

Key points:

- -O2 optimization: Full optimization (important for hashing performance in sha256_sw.c).
- Cross-compile: Uses aarch64-linux-gnu-gcc to build for the KV260's ARM Cortex-A53.
- Two targets: pdsa_main (production) and pdsa_miner (standalone CSR test tool).
- No external dependencies: -lm not needed (no math.h functions used in current code).
- No network libraries: Stratum/pthread removed with pool code.

15. Complete File Reference

PS Software Files:

=====

pdsa_main.c (940 lines)

Main application. Contains:

- pdsa_ctx_t context struct
- signal handler (Ctrl+C)
- get_hashrate()
- read_result_hash()
- send_job_to_pl() (BTC midstate + KAS header paths)
- create_simulated_job()
- do_dpr_switch()
- feed_nist_to_pl() + run_nist_self_test()
- feed_nist_kat() + run_nist_kat()
- run_genesis_block_test()
- run_pdsa_dpr_self_test()
- log_event()
- main() with local mining loop

pdsa_hal.h (58 lines)

Hardware Abstraction Layer. Contains:

- pdsa_hal_t struct
- pdsa_hal_init()
- pdsa_csr_write()
- pdsa_csr_read()
- pdsa_hal_close()

pdsa_csr_map.h (51 lines)

Shared register definitions.

mining/sha256_sw.c + sha256_sw.h

Software SHA-256 reference implementation.

- sha256(), sha256d(), bitcoin_compute_midstate()

mining/block_header.h

Mining job struct (mining_job_t):

- header, midstate, target, merkle_tail, timestamp, nbits

Compilation

Cross-compile (from x86_64 Linux):

```
make CC=aarch64-linux-gnu-gcc
```

Output:

```
pdsa_main (ELF 64-bit ARM aarch64 executable)
```

```
pdsa_miner (ELF 64-bit ARM aarch64 executable)
```

Size Summary

pdsa_main.c	:	940 lines	(main)
sha256_sw.c	:	210 lines	(SHA-256)
TOTAL	:	1,150 lines	(C source)
pdsa_hal.h	:	58 lines	(HAL header)

pdsa_csr_map.h	:	51 lines	(CSR definitions)
block_header.h	:	18 lines	(mining_job_t struct)
sha256_sw.h	:	55 lines	(function prototypes)
TOTAL	:	182 lines	(headers)

GRAND TOTAL	:	1,332 lines	

-- End of PS Explanation --